

CaptionAI: AI Powered Debt Relief & Financial Recovery Platform

Project Description:

The AI Powered Debt Relief & Financial Recovery Platform is an intelligent web application that helps users manage debt, analyze their financial condition, and receive AI-powered financial guidance. The system enables users to store their financial information, monitor loans, calculate repayment strategies, and generate personalized debt settlement recommendations using Google's Gemini AI.

The platform integrates React.js for the frontend, FastAPI for backend services, SQLAlchemy ORM with SQLite database for data management, and Google Gemini API for AI-powered financial analysis. The application provides a secure, scalable, and user-friendly solution for individuals seeking financial recovery.

Scenarios

Scenario 1: A user enters multiple loan details.

The AI analyzes monthly income, expenses, and debt and recommends the best repayment strategy.

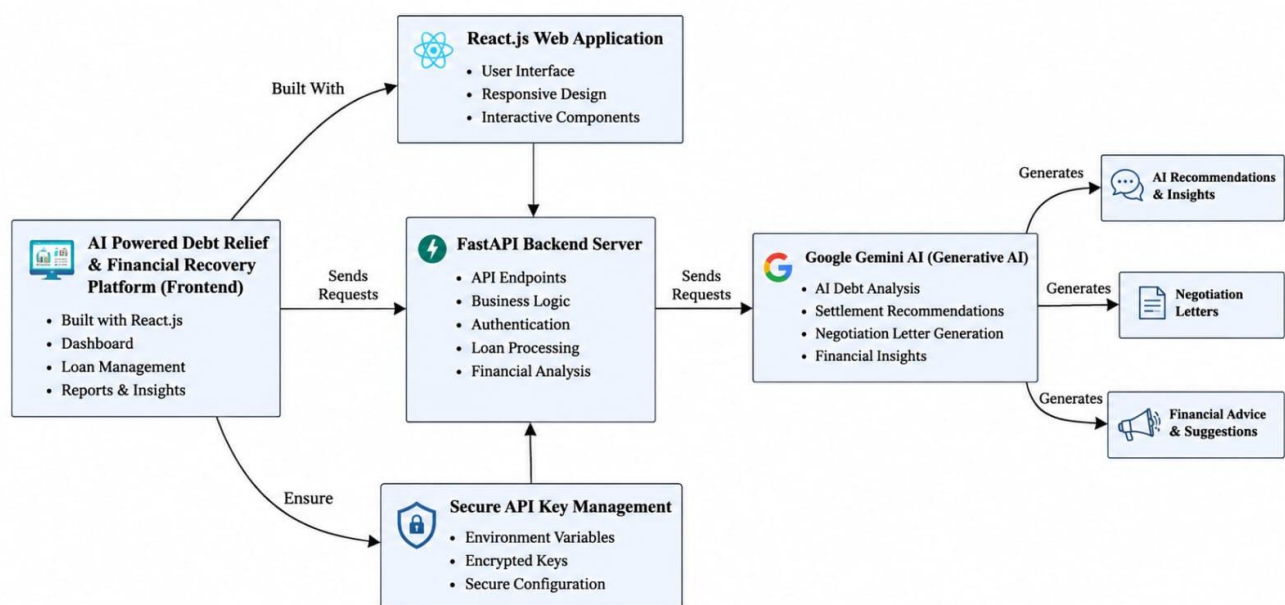
Scenario 2: A user uploads financial details.

Gemini AI generates personalized financial recovery advice and savings suggestions.

Scenario 3: A user wants to negotiate debt.

The AI automatically prepares a professional debt negotiation letter.

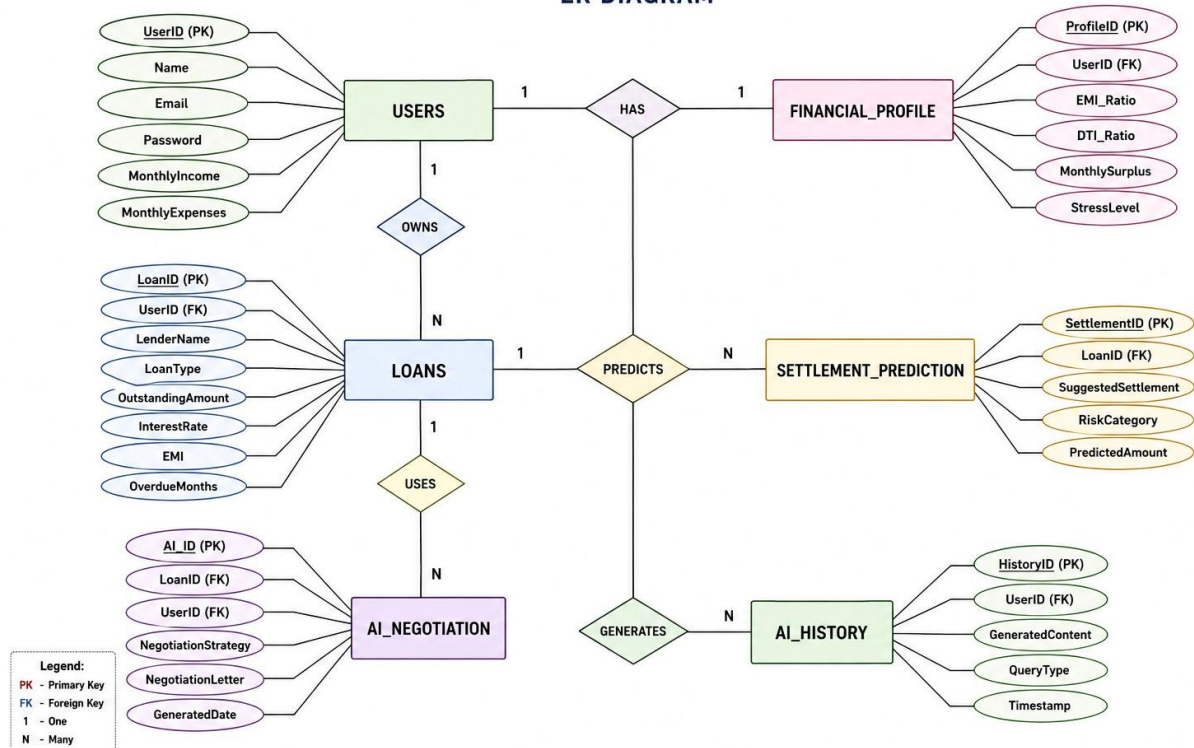
Technical Architecture:



Description: The application follows a three-tier architecture consisting of a React.js frontend, FastAPI backend, SQLite database, and Google Gemini AI integration. Users submit financial information through the frontend. The backend processes requests, stores data using SQLAlchemy ORM, communicates with Gemini AI for financial analysis, and returns personalized recommendations.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

AI POWERED DEBT RELIEF & FINANCIAL RECOVERY PLATFORM ER DIAGRAM



Pre-requisites:

- **Python 3.11+:** <https://www.python.org/downloads/>
- **FastAPI Documentation:** <https://fastapi.tiangolo.com/>
- **React.Js Documentaion:** <https://react.dev/>
- **Vite Documentation:** <https://vitejs.dev/>
- **Google Gemini API:** <https://aistudio.google.com/app/apikey>
- **SQLAlchemy ORM Documentation:** <https://www.sqlalchemy.org/>
- **Git Version Control:** <https://git-scm.com/doc>
- **GitHub Repository Hosting:** <https://github.com/>
- **Python Virtual Environment (venv):** <https://docs.python.org/3/library/venv.html>
- **Visual Studio Code:** <https://code.visualstudio.com/>
- **JWT Authentication Documentation:** <https://jwt.io/>
- **SQLite Documentation:** <https://www.sqlite.org/docs.html>

Project Workflow:

Milestone 1: Project Setup

Activities

- Install Python
- Install Node.js
- Create FastAPI Backend
- Create React Frontend
- Configure GitHub
- Install Dependencies

Milestone 2: Backend Development

Activities

- Create FastAPI APIs
- Create Database
- SQLAlchemy Models
- CRUD Operations
- User Authentication

Milestone 3: AI Integration

Activities

- Create Gemini API Key
- Configure .env
- Connect Gemini API
- Generate Financial Advice
- Generate Debt Settlement Letter

Milestone 4: Frontend Development

Activities

- Design Login Page
- Dashboard

- Loan Forms
- Financial Analysis Page
- Recommendation Page

Milestone 5: Testing

Activities

- API Testing
- Database Testing
- Frontend Testing
- AI Response Testing

Milestone 5: Deployment

Activities

- Deploy Backend
- Deploy Frontend
- GitHub
- Render
- Railway

Milestone 1: Model Selection and Architecture

In this milestone, we focused on selecting the most suitable Generative AI model and designing the overall architecture for the AI Powered Debt Relief & Financial Recovery Platform. The objective was to identify an AI model capable of providing personalized financial guidance, debt repayment recommendations, financial health analysis, and debt settlement letter generation. We selected Google Gemini because of its advanced reasoning capabilities, fast response generation, and seamless integration with Python applications through the Google Generative AI SDK. Additionally, the system architecture was designed to ensure scalability, security, and efficient communication between the frontend, backend, database, and AI services.

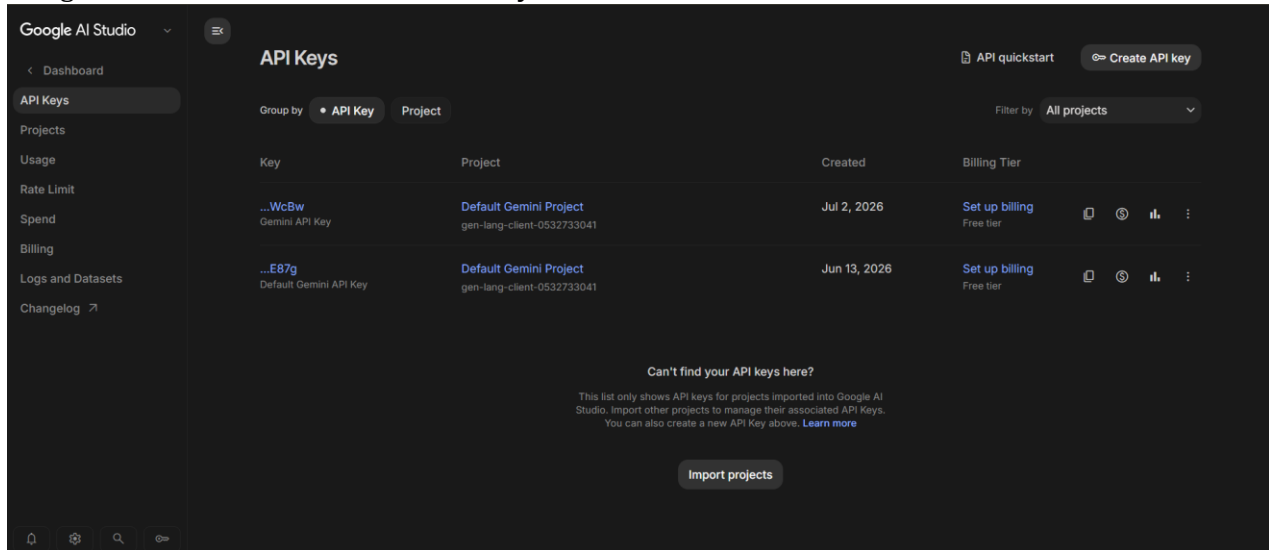
Activity 1.1: Generate a Google Gemini API Key

Before the application can communicate with the Google Gemini model, a valid Gemini API key is required. Follow the steps below to generate and configure the API key securely.

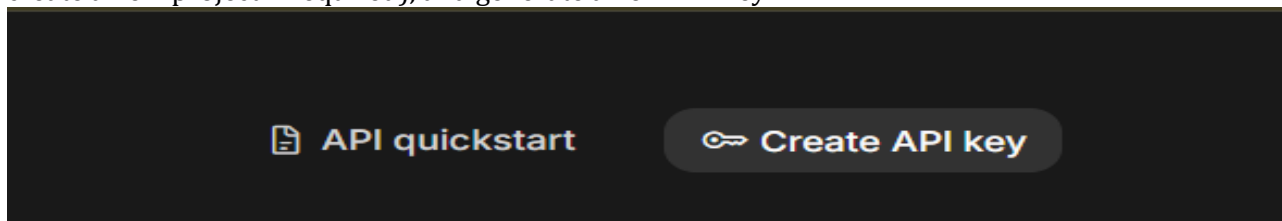
Step 1 — Open Google AI Studio:

Visit <https://aistudio.google.com/> and sign in using your Google account.

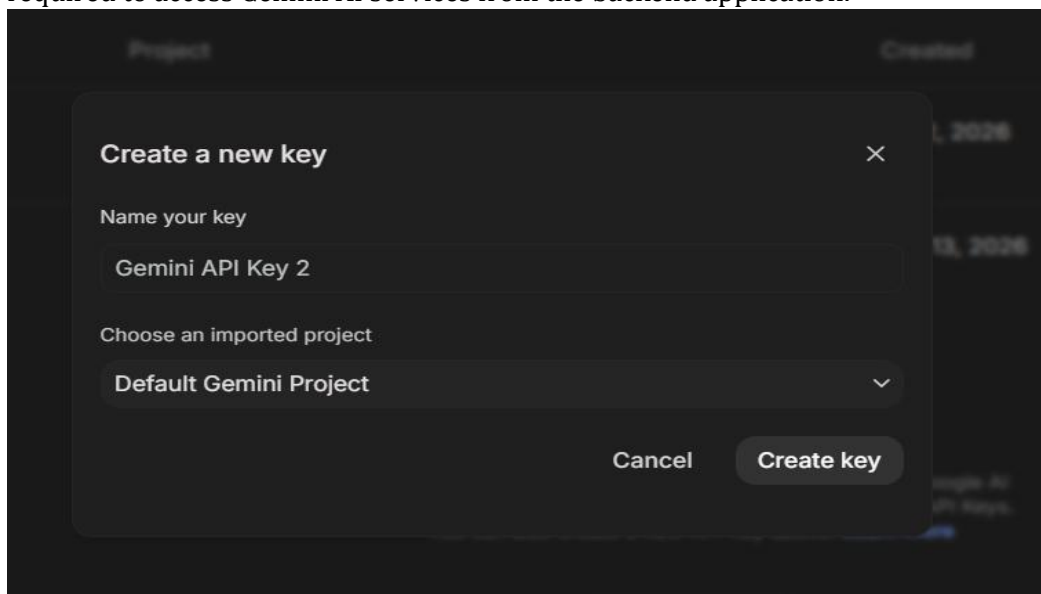
Step 2 — Navigate to API Keys: After signing in, open the **Get API Key** section from the left navigation menu or select **Create API Key**.



Step 3 — Create a New API Key: Click on **Create API Key**, select your Google Cloud project (or create a new project if required), and generate a new API key.



Step 4 — API Key: Copy the generated API key immediately and store it safely. The API key will be required to access Gemini AI services from the backend application.



Step 5 — Configure the API Key: Create a `.env` file in the backend project directory and add the following environment variable.

GEMINI_API_KEY=your_generated_api_key_here

Step 6 — Load the API Key: The FastAPI application loads the API key automatically using `python-dotenv`.

```
from dotenv import load_dotenv
import os

load_dotenv()

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
```

Important — Never upload the `.env` file or API key to GitHub. Add the following entry to the `.gitignore` file.

Activity 1.2: Research and Select the Appropriate Generative AI Model

The following research activities were performed before selecting the AI model.

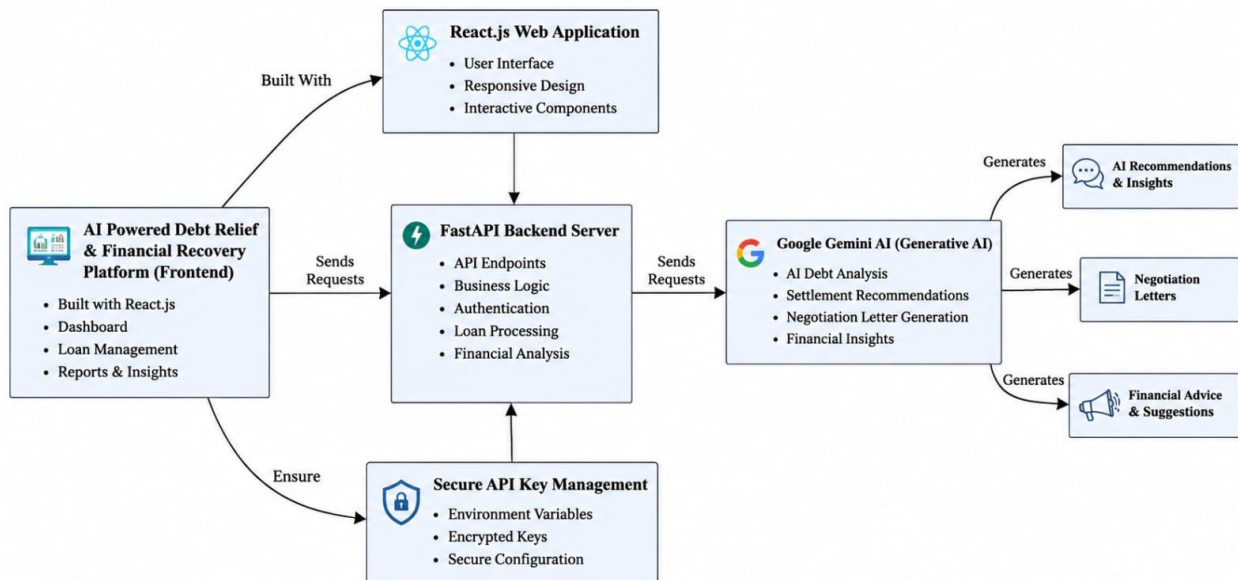
- **Understand the Project Requirements:** The application requires an AI model capable of analyzing financial information, generating debt repayment strategies, preparing settlement letters, answering financial queries, and providing personalized financial recovery recommendations.
- **Explore Google Gemini Documentation:** The Google Gemini documentation was studied to understand the available models, API features, token limits, pricing, response quality, and integration methods
- **Evaluate Model Performance:** Different Gemini models were compared based on:
 - Financial reasoning capability
 - Response accuracy
 - Processing speed
 - Cost efficiency
 - API integration support
 - Ability to generate structured financial recommendations
- **Select the Optimal Model:** The **Gemini 2.5 Flash** model was selected because it provides fast response generation, excellent reasoning capability, lower latency, and efficient processing for financial analysis applications. It is suitable for generating personalized debt management advice while maintaining high accuracy and performance.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** A three-tier architecture was designed consisting of:
 - React.js Frontend

- FastAPI Backend
- SQLite Database
- Google Gemini AI

Technical Architecture:



- **Frontend Responsibilities:** The React.js frontend provides an interactive user interface where users can:
 - Register and login
 - Add financial information
 - Manage loans
 - View financial reports
 - Request AI-powered recommendations
 - Generate debt settlement letters
- **Backend Responsibilities:** The FastAPI backend performs the following operations:
 - Receives API requests from the frontend
 - Validates user input
 - Performs business logic
 - Stores and retrieves data from SQLite using SQLAlchemy ORM
 - Communicates securely with Google Gemini API
 - Returns AI-generated financial recommendations to the frontend
- **Database Responsibilities:** SQLite stores all application data including:
 - User Information
 - Loan Details
 - Financial Profiles
 - AI Recommendation History

- Settlement Records
- SQLAlchemy ORM is used for efficient database management.

AI Integration

Google Gemini API is integrated into the backend to provide intelligent financial services such as:

- Financial health analysis
- Debt repayment planning
- Budget recommendations
- Personalized financial advice
- Debt settlement letter generation
- AI-powered financial guidance

The backend sends financial information to Gemini and returns structured AI responses to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure that Python 3.12 (or later) and pip are installed on your system.

```
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform> python --version
Python 3.14.2
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform> pip --version
pip 25.0.1 from C:\Program Files\Python314\Lib\site-packages\pip (python 3.14)
```

- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform> cd "Project Development Phase\backend"
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform\Project Development Phase\backend> python -m venv venv
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform\Project Development Phase\backend> venv\Scripts\activate
(venv) PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform\Project Development Phase\backend> pip install -r requirements.txt
```

- **Install FastAPI:** Install FastAPI for developing REST APIs.

```
(venv) PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform\Project Development Phase\backend> pip install fastapi
```

- **Install Uvicorn:** Install Uvicorn ASGI server for running the FastAPI application.

```
(venv) PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform\Project Development Phase\backend> pip install uvicorn
```

- **Install SQL Alchemy:**

```
(myenv) D:\projects> pip install sqlalchemy
```

- **Install Google Generative AI SDK:** Install the Google Generative AI Python SDK for communicating with Gemini AI

```
(myenv) D:\projects> pip install google-generativeai
```

- **Configure the API Key:** Create a .env file inside the backend folder and add the Gemini API Key:

```
GEMINI_API_KEY=your_api_key_here
```

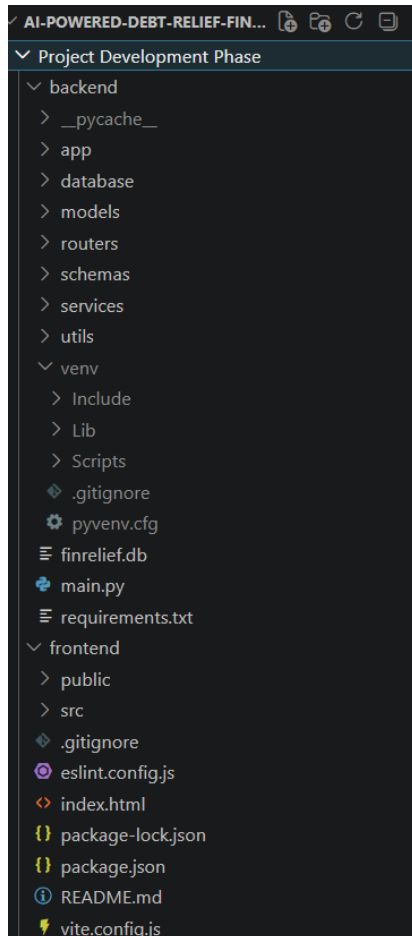
- **Install Frontend Dependencies:** Navigate to the frontend directory and install all required Node.js packages.


```
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform> cd "Project Development Phase/frontend"
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform\Project Development Phase\frontend> npm install
```

- **Install React Router:** React Router is used for navigation between application pages.

```
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform> cd "Project Development Phase/frontend"
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform\Project Development Phase\frontend> npm install react-router-dom
```

- **Set Up Application Structure:** Organize the project into modular folders for backend APIs, frontend components, database models, services, and utilities.



Milestone 2: Core Functionalities Development

In this milestone, the core functionalities of the **AI Powered Debt Relief & Financial Recovery Platform** were developed. The application enables users to register and log in securely, manage loan details, analyze financial health, receive AI-powered financial recommendations, generate debt settlement letters, and maintain a history of AI-generated responses. The backend APIs were implemented using FastAPI, while the frontend was developed using React.js for an interactive and responsive user experience.

Activity 2.1: Develop Core Financial Management Features

The following core features were implemented as part of the financial recovery platform.

User Authentication

- Developed secure user registration and login functionality.
- Passwords are encrypted before storing them in the database.
- User authentication is managed using secure backend APIs.

Loan Management

- Users can add multiple loan records.
- Loan details include lender name, loan amount, outstanding balance, interest rate, EMI, overdue months, and due date.
- Users can update and manage their loan information efficiently.

Financial Profile Management

- Users can enter their monthly income and monthly expenses.
- The application calculates the user's financial capacity for debt repayment.
- Financial profile information is stored securely in the SQLite database.

AI Financial Recommendation Generator

- Financial information and loan details are sent to the Google Gemini API.
- Gemini analyzes the user's financial condition.
- Personalized debt repayment strategies and financial recovery recommendations are generated.

Debt Settlement Letter Generator

- The system automatically generates professional debt settlement and negotiation letters using Google Gemini AI.
- Generated letters can be copied and used for communication with lenders.

AI Recommendation History

- Every AI-generated recommendation and settlement letter is stored.
- Users can review previously generated recommendations through the history module.

Activity 2.2: Implement the FastAPI Backend

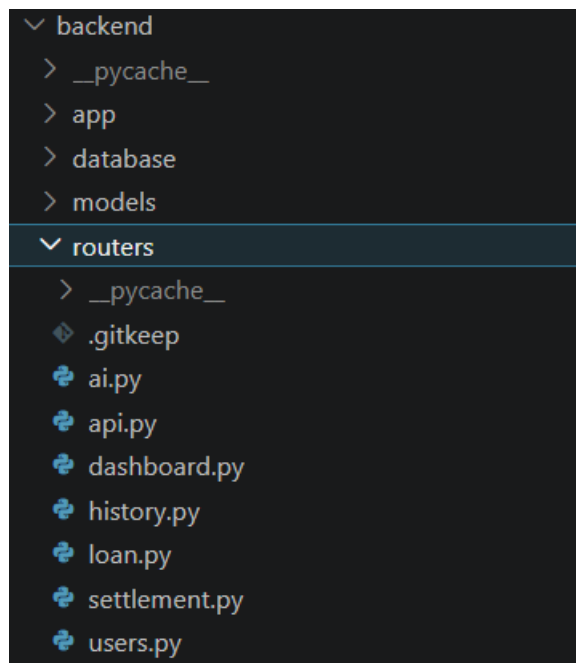
Define API Routes :

FastAPI routers were created to organize application functionalities into separate modules.

The backend includes dedicated API routes for:

- User Authentication
- Loan Management
- Financial Profile
- Dashboard
- AI Recommendation

- Settlement Letter Generation
- Recommendation History



Process User Input:

The React frontend collects user information through interactive forms.

The backend receives and validates the following information:

- User Registration Details
- Login Credentials
- Loan Information
- Monthly Income
- Monthly Expenses
- Financial Analysis Requests

Input validation ensures that mandatory fields are completed before processing.

Database Operations

SQLAlchemy ORM is used to interact with the SQLite database.

The backend performs:

- Create
- Read
- Update
- Delete (CRUD)

operations on:

- Users
- Loans
- Financial Profiles

- Settlement Records
- AI History

Integrate Google Gemini API

The backend communicates with Google Gemini AI to generate intelligent financial recommendations.

The following process is followed:

1. Receive user financial information.
2. Prepare a structured AI prompt.
3. Send the prompt to Google Gemini API.
4. Receive AI-generated financial advice.
5. Return the response to the React frontend.
6. Store AI history in the database.

Milestone 3: Main Application (main.py) Development

Milestone 3 focuses on developing the **main.py** file, which serves as the entry point of the **AI Powered Debt Relief & Financial Recovery Platform**. The file initializes the FastAPI application, configures middleware, establishes database connectivity, registers API routers, and enables communication between the frontend and backend services.

Activity 3.1: Writing the Main Application Logic in main.py

Initialize the FastAPI Application

The application begins by creating a FastAPI instance that serves as the backend server.

Features configured include:

- Application title
- API version
- API documentation
- Startup configuration



```
main.py ×
Project Development Phase > backend > main.py > ...
21
22 app = FastAPI(
23     title="FinRelief AI",
24     version="0.1.0"
25 )
```

Configure CORS Middleware:

To allow the React frontend to communicate with the FastAPI backend, Cross-Origin Resource Sharing (CORS) middleware is configured.

```
main.py X
Project Development Phase > backend > main.py > ...
26
27 app.add_middleware(
28     CORSMiddleware,
29     allow_origins=[
30         "http://localhost:5173"
31     ],
32     allow_credentials=True,
33     allow_methods=["*"],
34     allow_headers=["*"],
35 )
```

This enables secure communication between the frontend and backend during development.

Initialize Database Connection:

The application imports the SQLAlchemy database engine and automatically creates database tables during startup.

```
from database.connection import Base, engine

37 # Create database tables
38 Base.metadata.create_all(bind=engine)
```

This initializes the SQLite database used to store:

- User information
- Loan records
- Financial profiles
- Settlement records
- AI history

Import Database Models:

All database models are imported into main.py before table creation.

The imported models include:

- User
- Loan
- FinancialProfile
- SettlementRecord
- AIHistory

This ensures SQLAlchemy creates the required tables automatically.

```
main.py X
Project Development Phase > backend > main.py > ...
6 # Import all models
7 from models.user import User
8 from models.loan import Loan
9 from models.financial_profile import FinancialProfile
10 from models.settlement import SettlementRecord
11 from models.ai_history import AIHistory
```

Register API Routers:

The application organizes functionality into multiple routers for modular development.

The following routers are registered:

```
main.py X
Project Development Phase > backend > main.py > ...
13 # Import routers
14 from routers.users import router as user_router
15 from routers.settlement import router as settlement_router
16 from routers.ai import router as ai_router
17 from routers.api import router as api_router
18 from routers.loan import router as loan_router
19 from routers.dashboard import router as dashboard_router
20 from routers.history import router as history_router
```

These routers manage:

- User Authentication
- Loan Management
- Financial Profile
- Dashboard
- AI Recommendation
- Settlement Letter Generation
- Recommendation History

Root Endpoint:

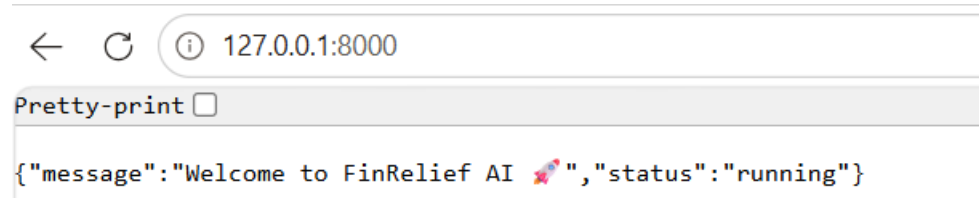
A default API endpoint is created to verify that the backend server is running correctly.

```
main.py X
Project Development Phase > backend > main.py > ...
50 @app.get("/")
51 def read_root():
52     return {
53         "message": "Welcome to FinRelief AI 🚀",
54         "status": "running"
55     }
```

Opening

http://127.0.0.1:8000/

returns the backend status.



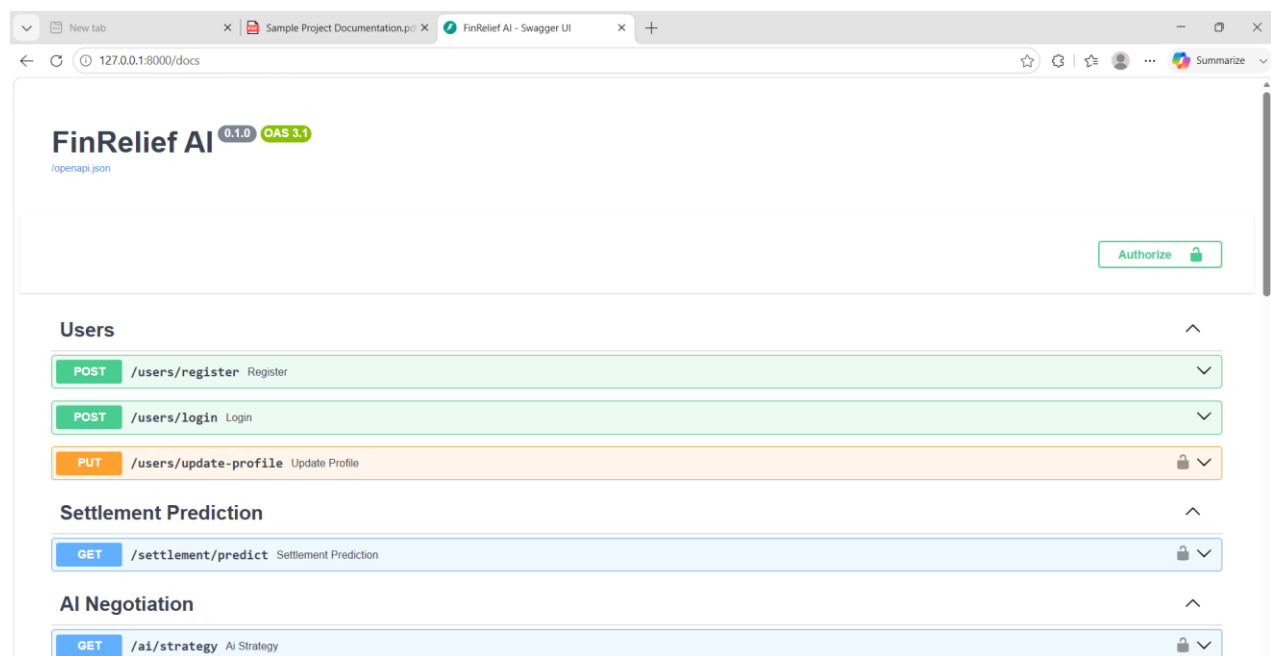
API Documentation

FastAPI automatically generates interactive API documentation.

Available endpoints include:

http://127.0.0.1:8000/docs

Swagger UI allows developers to test APIs without writing frontend code.



AI Integration

When the frontend sends financial information, the backend:

1. Receives the request.
2. Validates the input.
3. Retrieves financial information.
4. Sends the structured prompt to Google Gemini AI.
5. Receives the generated financial recommendation.
6. Returns the response to the frontend.

7. Stores the AI response in the database.

Error Handling

The backend validates all incoming requests before processing.

Validation includes:

- Empty input checking
- Missing required fields
- Invalid financial values
- Database exceptions
- Google Gemini API errors

Appropriate HTTP status codes and error messages are returned to the frontend.

Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing a responsive, user-friendly frontend for the **AI Powered Debt Relief & Financial Recovery Platform**. The frontend is built using **React.js**, providing an interactive interface where users can manage financial information, analyze debts, receive AI-powered recommendations, and generate settlement letters. The application emphasizes usability, responsiveness, and seamless communication with the FastAPI backend.

Activity 4.1: Designing and Developing the User Interface

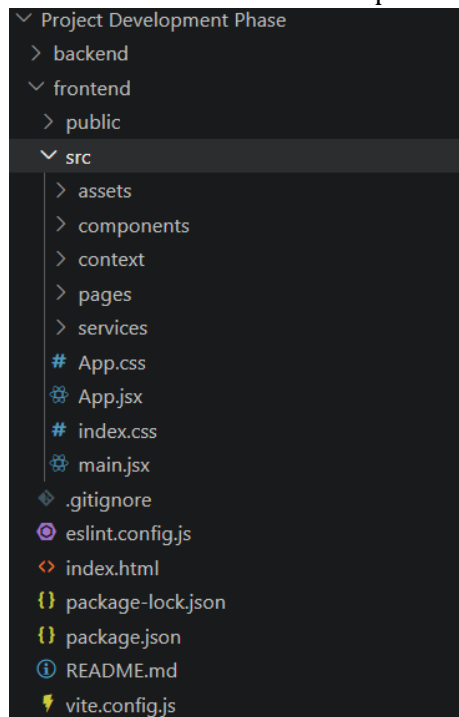
Develop the React Application Structure

The frontend is organized using reusable React components and React Router for navigation.

The project structure includes:

- Components
- Pages
- Services
- Context
- Hooks
- Utils

This modular architecture improves maintainability and code reusability.

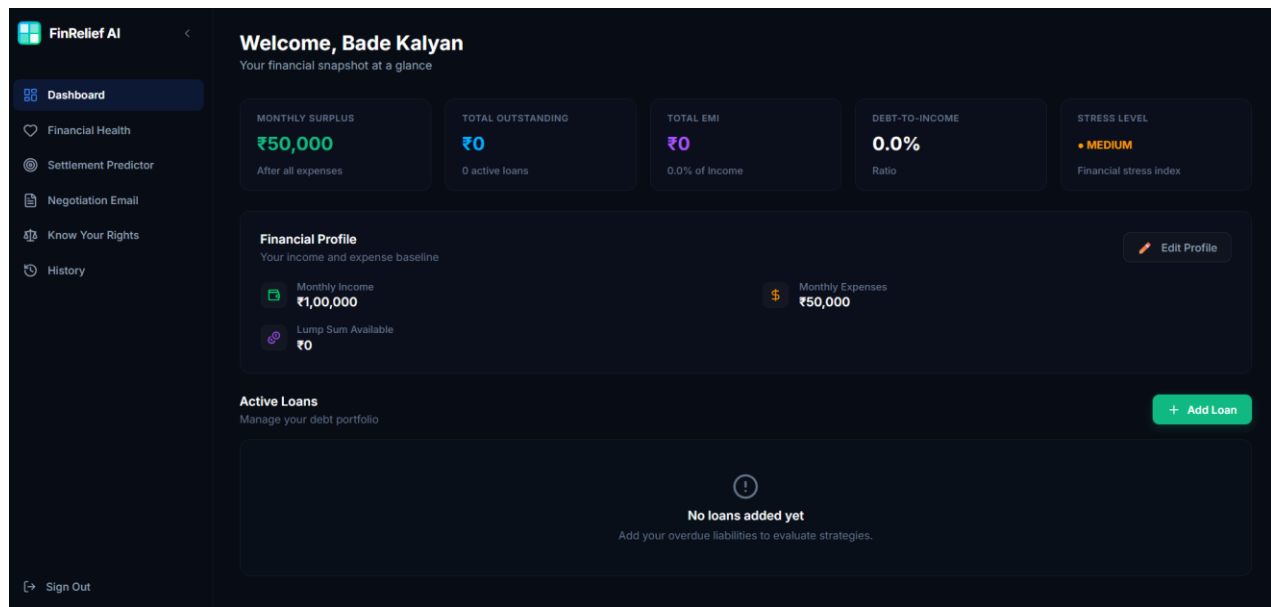


Create Responsive User Interfaces

Develop responsive pages for the following modules:

- Login Page
- Registration Page
- Dashboard
- Financial Profile
- Loan Management
- AI Recommendation
- Settlement Letter Generator
- Recommendation History

Each page provides a clean and intuitive interface for users to manage their financial information efficiently.



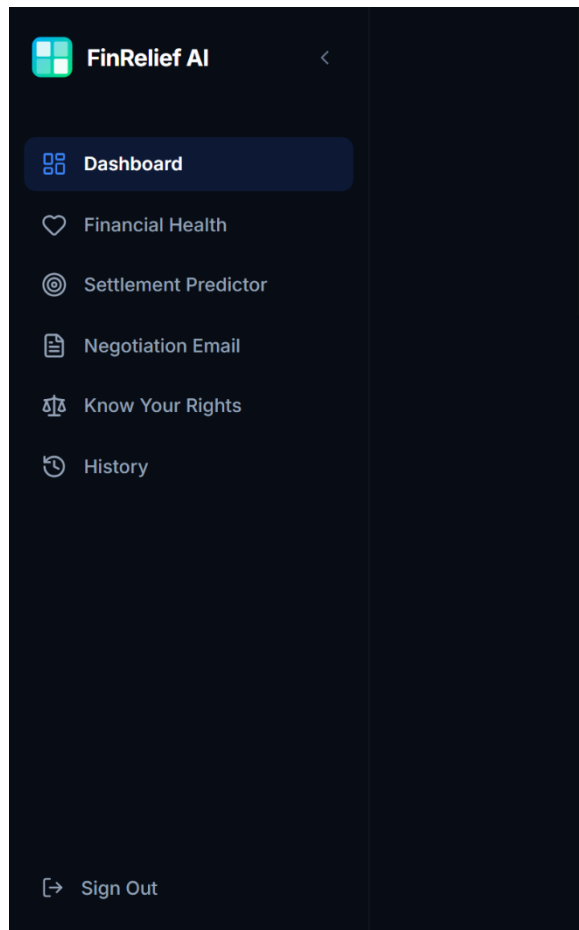
Design Using CSS:

Develop responsive styling using CSS.

Features include:

- Responsive layouts
- Cards for financial information
- Tables for loan details
- Forms with validation
- Navigation sidebar
- Mobile-friendly design

Media queries are used to ensure compatibility across desktop, tablet, and mobile devices.



Create Reusable Components

The frontend uses reusable React components including:

- Navigation Bar
- Sidebar
- Loan Card
- Financial Summary Card
- AI Recommendation Card
- Settlement Letter Viewer
- Loading Spinner
- Alert Messages

This improves consistency across the application.

Dashboard Interface

Design an interactive dashboard displaying:

- Total Debt

- Monthly Income
- Monthly Expenses
- Savings
- Credit Score
- Financial Health Summary

Charts and summary cards help users understand their financial status quickly.

Loan Management Interface

Create forms that allow users to:

- Add Loan
- Edit Loan
- Delete Loan
- View Loan Details

Loan information includes:

- Loan Type
- Principal Amount
- Interest Rate
- Monthly EMI
- Remaining Balance
- Due Date

AI Recommendation Interface

Design a dedicated page where users can:

- View financial profile
- Request AI-generated recommendations
- Display personalized debt recovery suggestions
- Save recommendation history

Recommendations are displayed in well-formatted cards for better readability.

Settlement Letter Generator

Develop an interface that allows users to:

- Select a loan
- Generate settlement letters
- Preview generated letters
- Download or copy settlement content

Activity 4.2: Frontend Integration with Backend

Handle API Requests

The React frontend communicates with the FastAPI backend using **Axios**.

API requests include:

- User Authentication
- Financial Profile
- Loan Management
- AI Recommendation
- Settlement Letter Generation
- Recommendation History

Each request exchanges JSON data with the backend.

eg:

```
axios.get("/api/dashboard")
```

```
axios.post("/api/loan")
```

```
axios.post("/api/ai/recommend")
```

Form Validation

Before submitting user data, the frontend validates:

- Required fields
- Numeric values
- Loan amount
- Interest rate
- Monthly income
- Email format

Validation errors are displayed immediately to improve user experience.

Dynamic Rendering

React dynamically updates the interface after receiving backend responses.

Examples include:

- Updated dashboard statistics
- Newly added loans
- AI recommendations
- Settlement letters
- Recommendation history

No page refresh is required.

Loading Indicators

During backend processing, the application displays:

- Loading Spinner

- Processing Messages
- Disabled Submit Buttons

This informs users that the request is being processed.

Error Handling

The frontend displays meaningful error messages when:

- Backend server is unavailable
- Invalid input is submitted
- Google Gemini API fails
- Database operation fails

This improves reliability and user experience.

Responsive Navigation

React Router enables smooth navigation between pages without reloading the application.

Navigation includes:

- Dashboard
- Profile
- Loans
- AI Recommendation
- Settlement Letter
- History

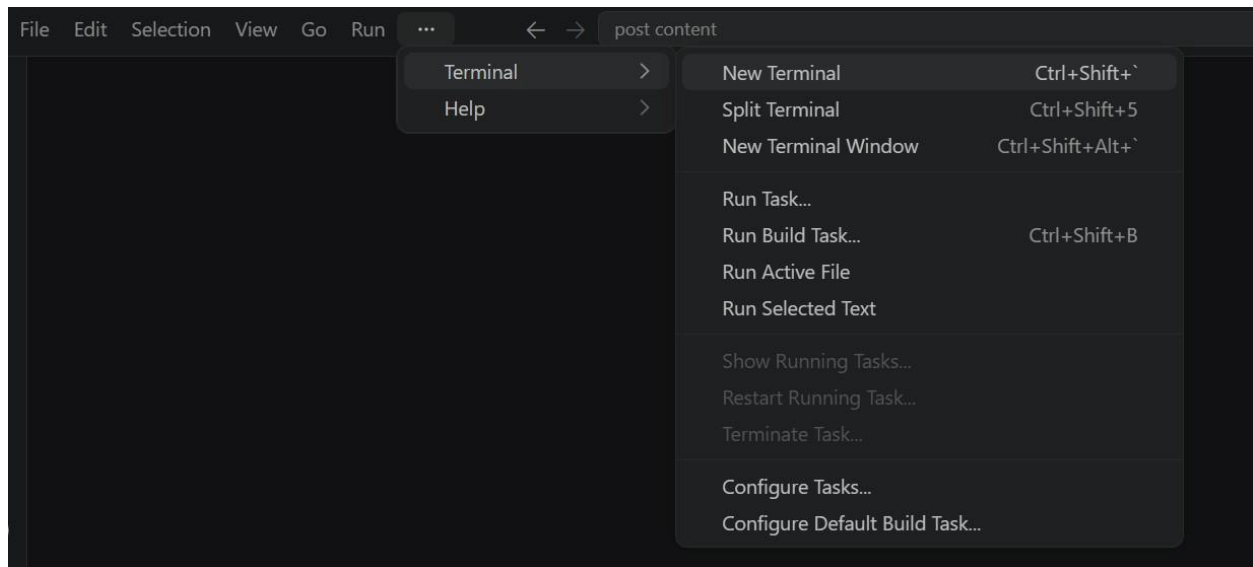
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the **AI Powered Debt Relief & Financial Recovery Platform**. The application is first tested in the local development environment to verify all functionalities, and then deployed to cloud platforms for public access. The backend is hosted using **Render**, while the frontend is deployed using **Vercel**. This deployment enables users to securely access the application from anywhere through a web browser.

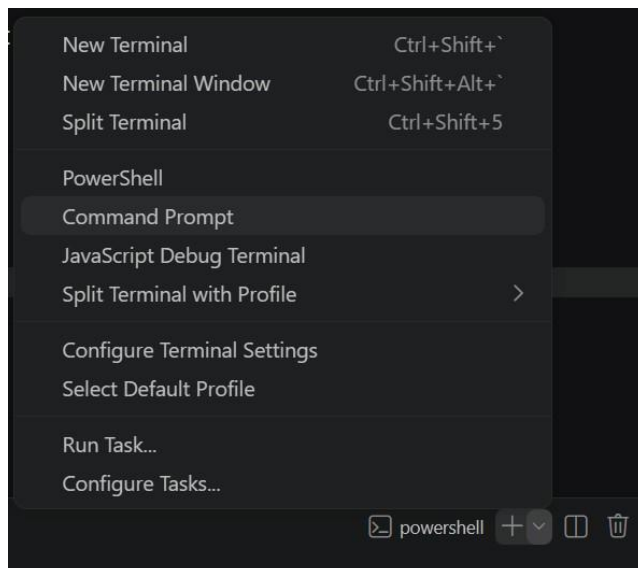
Activity 5.1: Preparing the Application for Local Deployment

Set Up the Backend Environment:

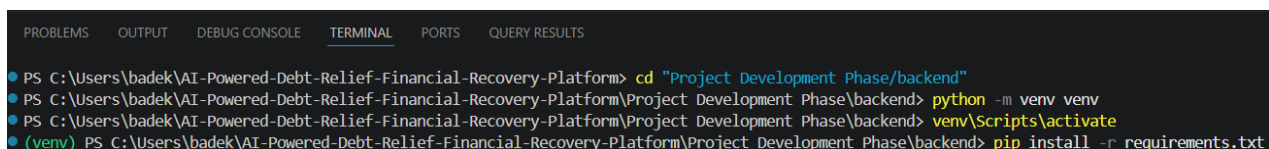
- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.



Configure Environment Variables:

- Create a **.env** file inside the backend directory and add the Google Gemini API key.
- `GOOGLE_API_KEY=your_google_gemini_api_key`

- The application automatically loads the environment variables using the **python-dotenv** package during startup.

Start the Backend Server:

Run the FastAPI application using Uvicorn.

```
uvicorn main:app --reload
```

Once the backend starts successfully, open the browser and navigate to:

<http://127.0.0.1:8000/docs>

The Swagger API documentation confirms that all backend APIs are functioning correctly.

Start the Frontend Application:

Navigate to the frontend folder.

Install all frontend dependencies.

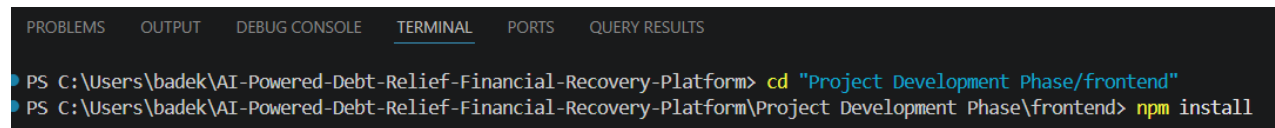
```
npm install
```

Run the React application.

```
npm run dev
```

Open the browser and access:

<http://localhost:5173>



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  QUERY RESULTS
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform> cd "Project Development Phase/frontend"
PS C:\Users\badek\AI-Powered-Debt-Relief-Financial-Recovery-Platform\Project Development Phase\frontend> npm install
```

Activity 5.2: Local Testing and Verification

Before deploying the application to the cloud, all modules were thoroughly tested in the local environment.

The following functionalities were verified successfully:

- User Registration and Login
- Secure JWT Authentication
- Dashboard Data Retrieval
- Loan CRUD Operations
- Financial Profile Management
- AI-Powered Debt Negotiation Strategy
- Debt Settlement Letter Generation
- Recommendation History Storage
- Database Operations using SQLite

The backend APIs were verified using the FastAPI Swagger Documentation, while the frontend was tested using the React development server.

Activity 5.3: Cloud Deployment

The application was deployed using modern cloud hosting platforms to enable public access.

Backend Deployment using Render

The backend source code was pushed to GitHub and connected to the Render cloud platform.

The deployment configuration included:

- Root Directory: **Project Development Phase/backend**

- Build Command:

```
pip install -r requirements.txt
```

- Start Command:

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

The Google Gemini API key was securely configured using Render Environment Variables.

After successful deployment, the backend became publicly accessible through the Render URL.

Frontend Deployment using Vercel

The React frontend was deployed using the Vercel cloud platform.

The deployment process included:

- Connecting the GitHub repository
- Selecting the frontend project directory
- Installing all frontend dependencies
- Building the React application
- Deploying the production build

The frontend was configured to communicate with the deployed Render backend through REST APIs.

CORS Configuration

To allow secure communication between the frontend and backend, Cross-Origin Resource Sharing (CORS) was configured in the FastAPI application.

The deployed Vercel domain was added to the list of allowed origins, enabling secure API communication between both cloud platforms.

CORS Configuration

To allow secure communication between the frontend and backend, Cross-Origin Resource Sharing (CORS) was configured in the FastAPI application.

The deployed Vercel domain was added to the list of allowed origins, enabling secure API communication between both cloud platforms.

Public Access Verification

After deployment, the application was successfully tested using the live cloud URLs.

The following functionalities were verified:

- User Registration
- User Login
- Dashboard Access
- Loan Management
- AI Recommendation Generation
- Settlement Letter Generation
- Recommendation History

The successful execution of all modules confirmed that both frontend and backend deployments were functioning correctly.

PUBLIC WEBSITE URL IS LIVE!

URL: <https://ai-powered-debt-relief-financial-re-livid.vercel.app/>

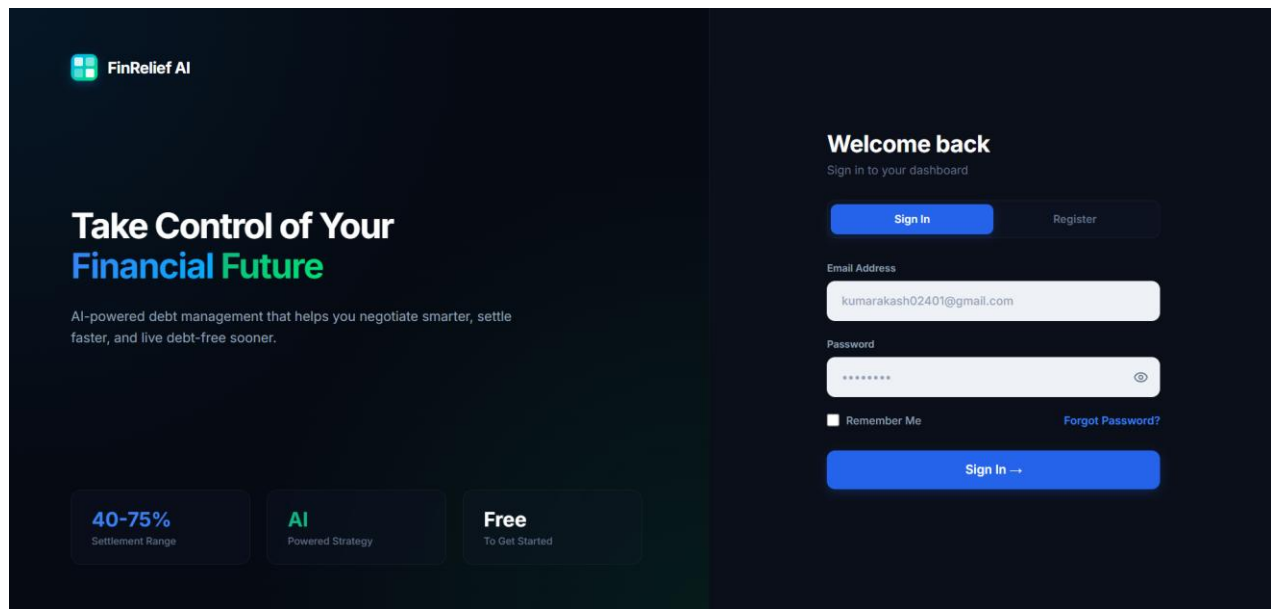
Important Notes:

- The backend is hosted on Render, while the frontend is hosted on Vercel.
- GitHub is used as the source repository for continuous deployment.
- Every new commit pushed to the main branch automatically triggers a new deployment on both Render and Vercel.
- The Google Gemini API Key is securely stored as an Environment Variable in Render and is never exposed in the source code.
- The application can be accessed publicly through the deployed cloud URLs without requiring any local installation.

Exploring the Website's Web Pages:

Login Page

Description: The Login page serves as the secure entry point to the AI Powered Debt Relief & Financial Recovery Platform. Users can log in using their registered email address and password. The interface provides input validation, secure authentication, and error handling to ensure only authorized users can access their financial information. Users who do not have an account can navigate to the Registration page, while existing users can securely access the Dashboard after successful authentication.

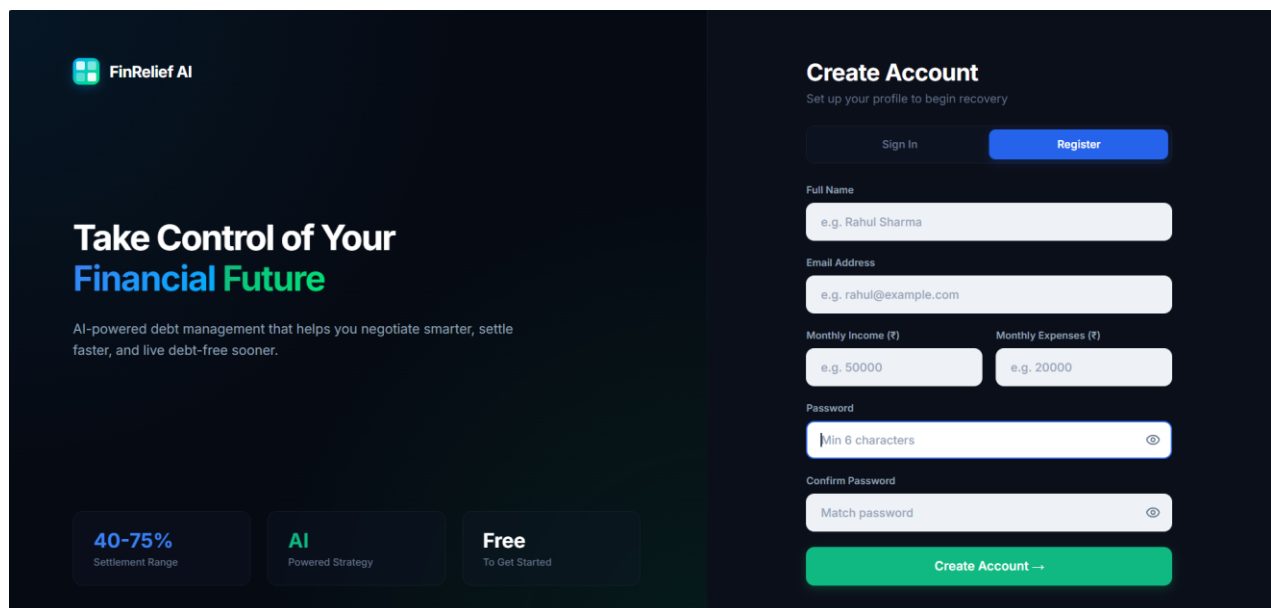


The image shows a sign-in page for 'FinRelief AI'. On the left, there's a hero section with the title 'Take Control of Your Financial Future' and a subtext 'AI-powered debt management that helps you negotiate smarter, settle faster, and live debt-free sooner.' Below this are three boxes: '40-75% Settlement Range', 'AI Powered Strategy', and 'Free To Get Started'. On the right, there's a 'Welcome back' section with a 'Sign in to your dashboard' link. It features a 'Sign In' button and a 'Register' button. Below these are input fields for 'Email Address' (with the example 'kumarakash02401@gmail.com') and 'Password' (with a masked password '*****'). There are also checkboxes for 'Remember Me' and a link for 'Forgot Password?'. A large blue 'Sign In -->' button is at the bottom.

Registration Page

Description:

The Registration page enables new users to create an account by providing their personal information, including name, email address, and password. The application validates user inputs before securely storing the information in the SQLite database. Passwords are encrypted to ensure data security and user privacy.

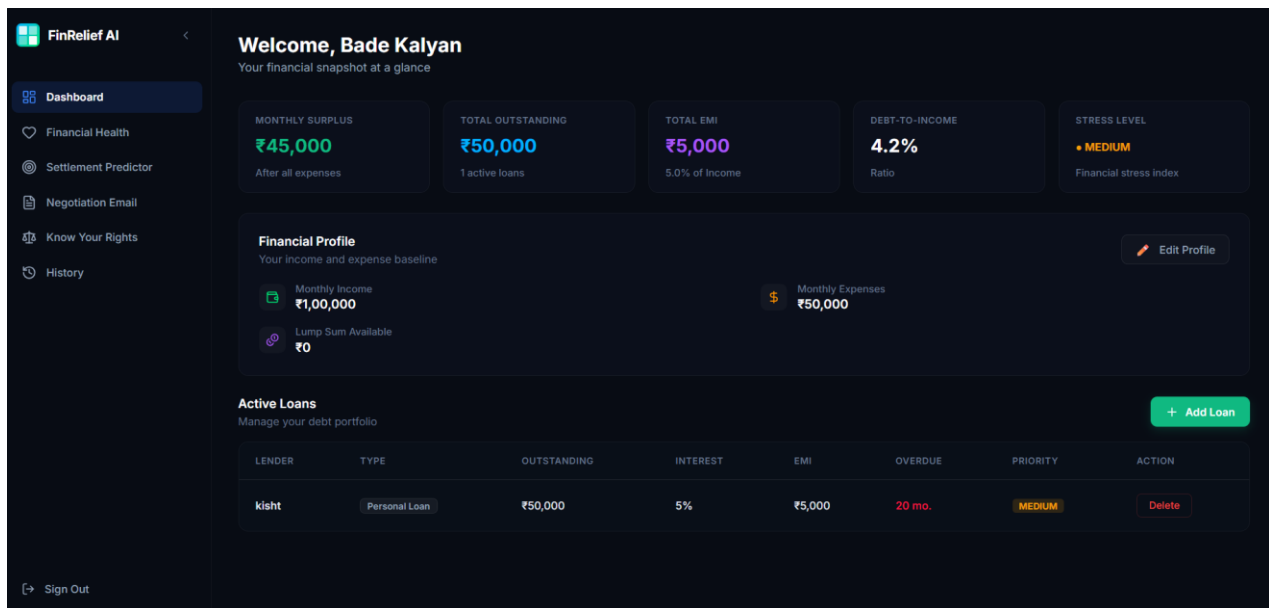


The image shows a 'Create Account' page for 'FinRelief AI'. On the left, it's identical to the sign-in page. On the right, there's a 'Create Account' section with a 'Set up your profile to begin recovery' link. It features a 'Sign In' button and a 'Register' button. Below these are input fields for 'Full Name' (with the example 'e.g. Rahul Sharma'), 'Email Address' (with the example 'e.g. rahul@example.com'), 'Monthly Income (₹)' (with the example 'e.g. 50000'), and 'Monthly Expenses (₹)' (with the example 'e.g. 20000'). There are also fields for 'Password' (with a hint 'Min 6 characters') and 'Confirm Password' (with a hint 'Match password'). A large green 'Create Account -->' button is at the bottom.

Dashboard

Description: The Dashboard provides an overview of the user's financial information. It displays key financial indicators such as total outstanding debt, monthly income, monthly expenses, savings,

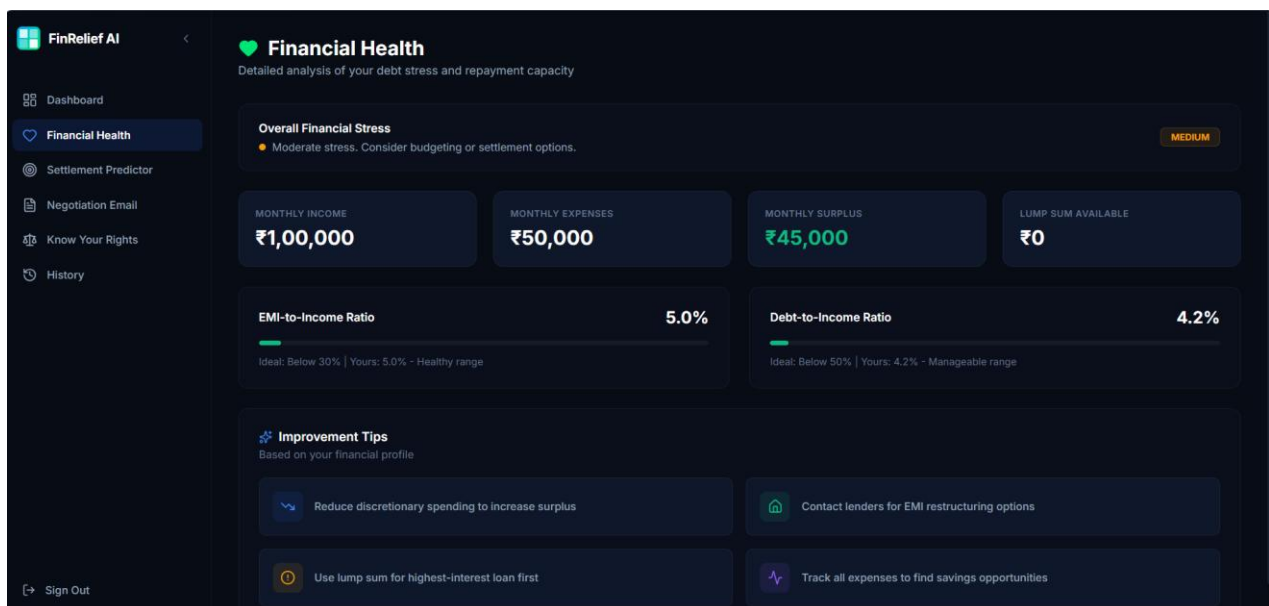
financial health summary, and recent activity. Users can easily navigate to different modules, including Loan Management, Financial Analysis, AI Recommendations, and Settlement Letter Generation.



Financial Health Page

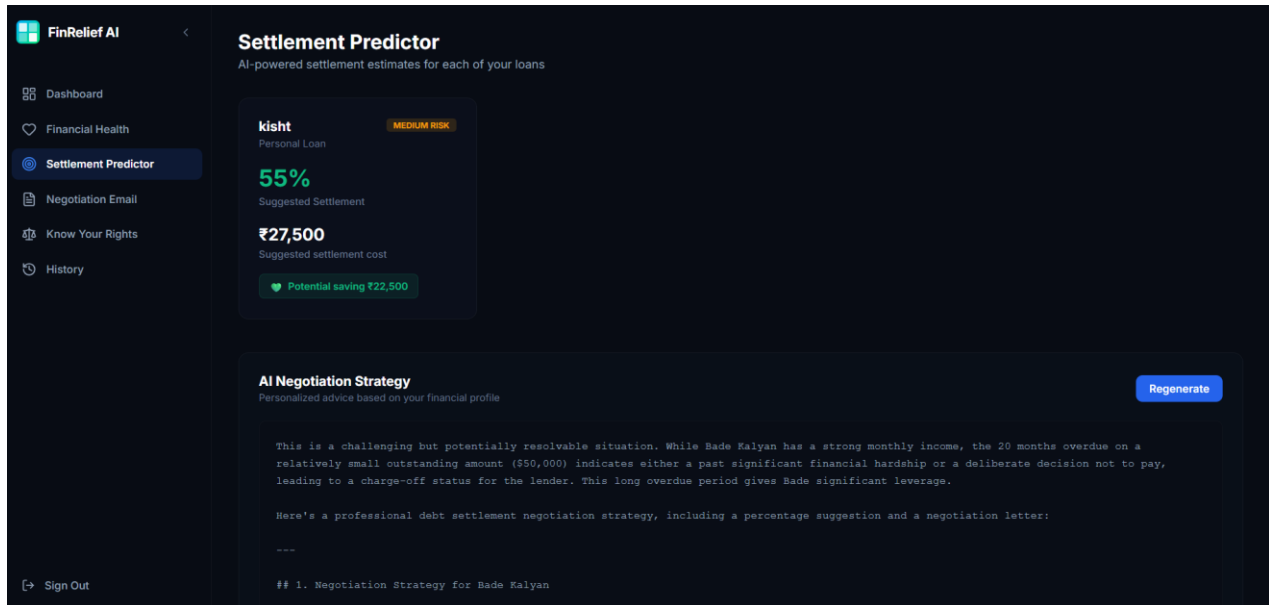
Description:

The Financial Health page provides users with a summary of their financial condition based on their monthly income, expenses, savings, and loan information. The system calculates important financial metrics and presents them in an easy-to-understand format, helping users better understand their financial stability and repayment capacity.



AI Recommendation Page

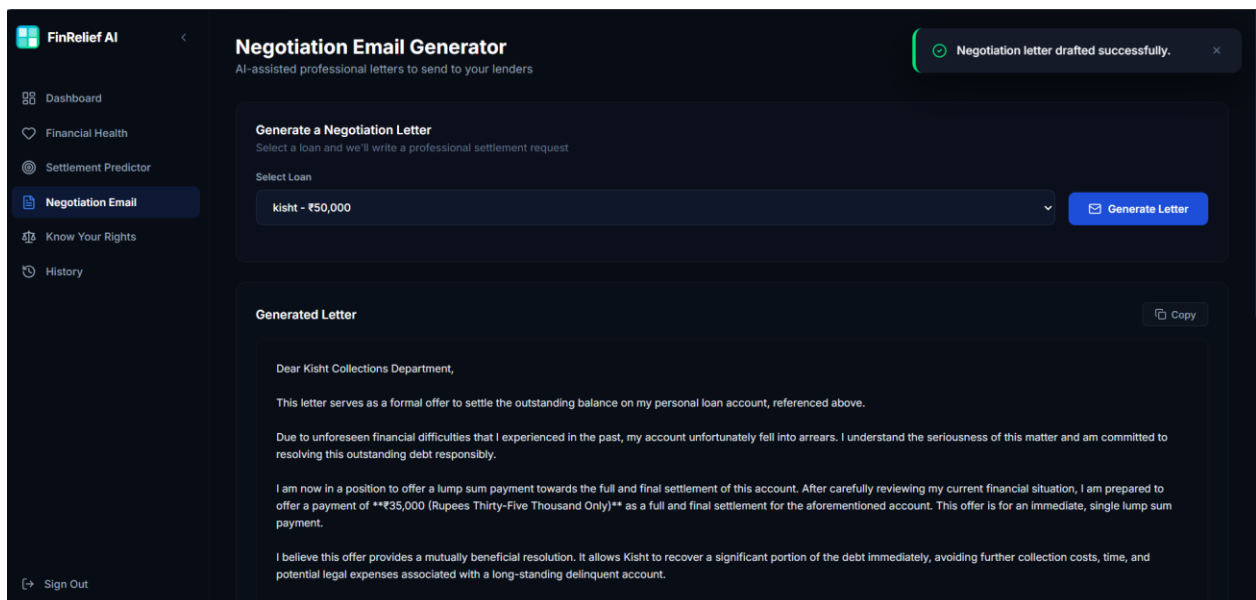
Description: The AI Recommendation page enables users to receive personalized financial advice using Google Gemini AI. Based on the user's financial profile and loan details, the system generates intelligent recommendations, repayment strategies, budgeting suggestions, and financial recovery plans. If the Gemini API is unavailable, the application automatically switches to the fallback recommendation engine to ensure uninterrupted functionality.



The screenshot shows the 'Settlement Predictor' section of the FinRelief AI app. The left sidebar contains navigation links: Dashboard, Financial Health, Settlement Predictor (active), Negotiation Email, Know Your Rights, and History. The main content area is titled 'Settlement Predictor' with the subtitle 'AI-powered settlement estimates for each of your loans'. It displays a card for a 'kisht' Personal Loan with a 'MEDIUM RISK' label. The card shows a '55%' Suggested Settlement and a '₹27,500' Suggested settlement cost, with a green button indicating 'Potential saving ₹22,500'. Below this is the 'AI Negotiation Strategy' section, which provides personalized advice based on the user's financial profile. The strategy text discusses a challenging but potentially resolvable situation for 'Bade Kalyan' with a 20-month overdue on a ₹50,000 loan, suggesting a charge-off status and a negotiation strategy. A 'Regenerate' button is located at the top right of this section. The bottom of the page shows a 'Sign Out' link.

Debt Settlement Letter Page

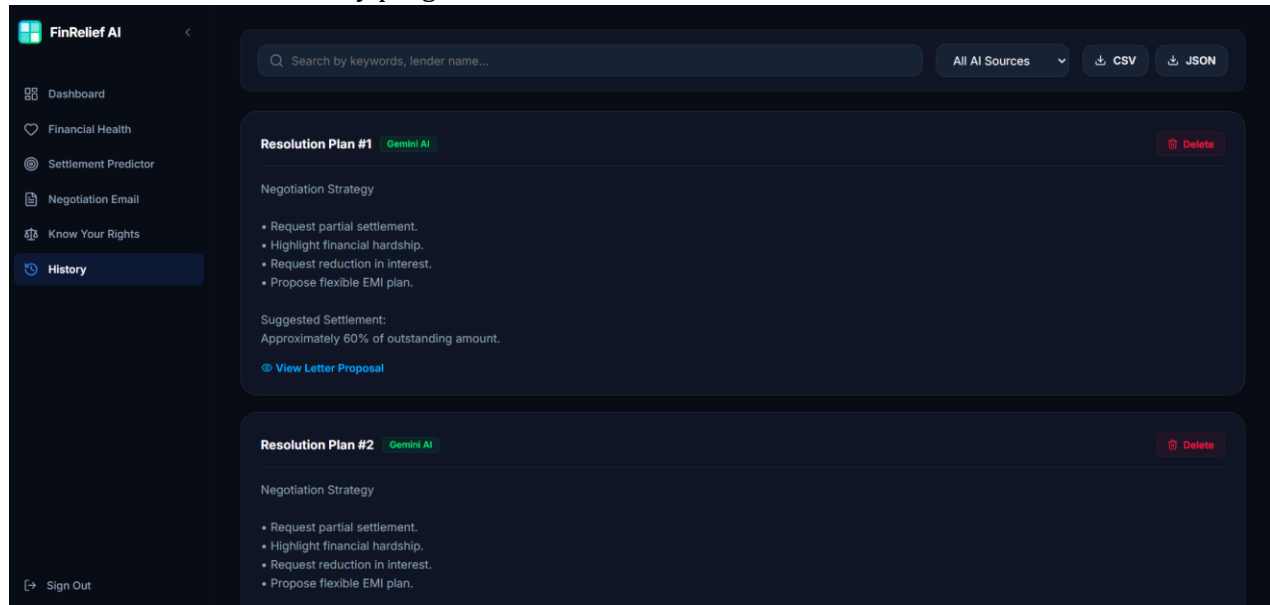
Description: The Debt Settlement Letter page generates professional debt negotiation letters using AI. Users can preview, copy, and use the generated settlement letter while communicating with lenders. The generated content is customized according to the user's financial information and selected loan details.



The screenshot shows the 'Negotiation Email Generator' section of the FinRelief AI app. The left sidebar is identical to the previous page. The main content area is titled 'Negotiation Email Generator' with the subtitle 'AI-assisted professional letters to send to your lenders'. A green notification banner at the top right states 'Negotiation letter drafted successfully.' Below this is the 'Generate a Negotiation Letter' section, which prompts the user to 'Select a loan and we'll write a professional settlement request'. A dropdown menu shows 'kisht - ₹50,000' and a 'Generate Letter' button. The 'Generated Letter' section displays a professional debt settlement letter addressed to 'Dear Kisht Collections Department'. The letter text includes a formal offer to settle the outstanding balance, a statement of financial difficulties, and a lump sum payment offer of ₹35,000. A 'Copy' button is located at the top right of the generated letter. The bottom of the page shows a 'Sign Out' link.

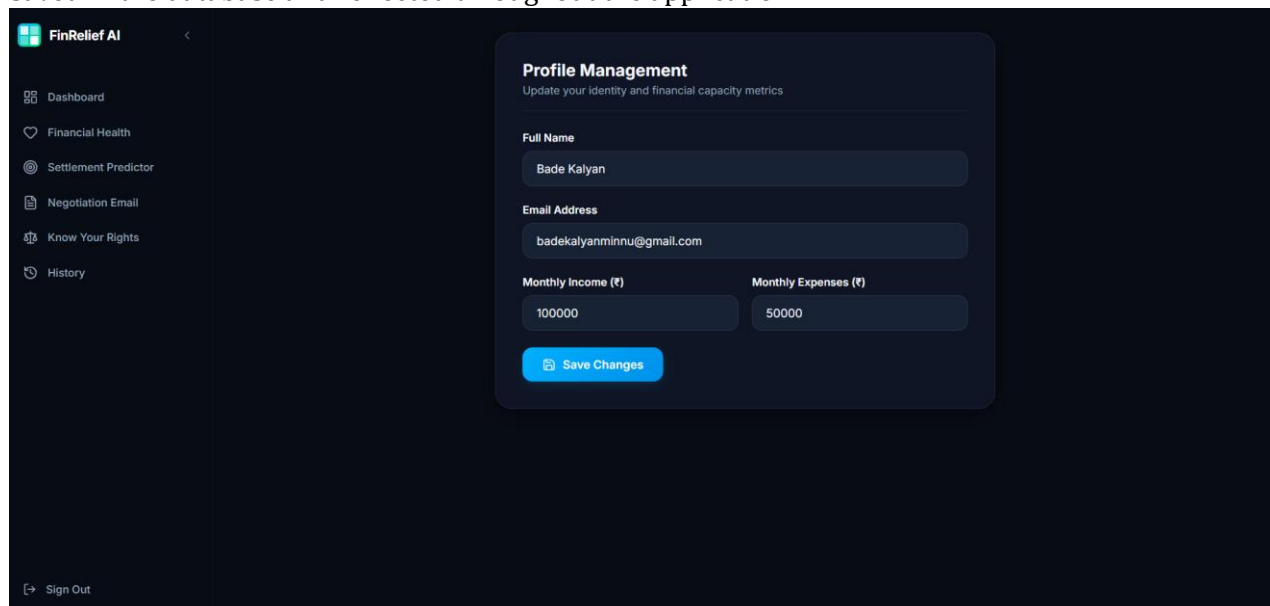
Recommendation History Page

Description: The Recommendation History page stores previously generated AI recommendations and settlement letters. Users can review past financial advice, revisit earlier recommendations, and track their financial recovery progress over time.



Profile Page

Description: The Profile page allows users to view and update their personal information, including their name, email address, monthly income, and monthly expenses. Profile updates are securely saved in the database and reflected throughout the application.



Conclusion:

The **AI Powered Debt Relief & Financial Recovery Platform** is an intelligent full-stack financial technology application developed using **React.js**, **FastAPI**, **SQLite**, **SQLAlchemy ORM**, and **Google Gemini AI**. The platform assists users in managing loans, analyzing financial health, generating personalized repayment strategies, and creating AI-powered debt settlement letters. The application emphasizes security, scalability, responsive user experience, and modular architecture, making it suitable for future enhancements and real-world financial assistance.

The project successfully demonstrates the integration of modern web technologies with Generative AI to solve practical financial challenges. Future enhancements may include cloud deployment, mobile application support, multi-language assistance, credit score prediction, banking API integration, email notifications, PDF report generation, and advanced AI-powered financial planning. These improvements will further enhance the platform's scalability, accessibility, and effectiveness in supporting users throughout their financial recovery journey.